

MINI PROJECT – II

STUDENT RECORD MANAGEMENT SYSTEM

Project Assignment – C Programming

Objective: Develop a menu-driven Student Record Management System in C using a **singly linked list**, dynamic memory allocation, modular programming, searching, sorting, and file handling.

Executable: The final application must be built as an executable named **student**.

1. Project Requirements

- The program must be menu-driven and should repeatedly display the main menu until the user chooses Exit.
- Student records must be maintained using a singly linked list.
- Each new student node must be dynamically allocated.
- The project must be divided into separate source modules.
- Student data must be saved in a file named **student.dat**.
- All dynamically allocated memory must be released before the program terminates.

2. Suggested Project Modules

Module	Responsibility
stud_main.c	Main function, menu and overall program flow.
stud_add.c	Add a new student record.
stud_del.c	Delete a student record.
stud_show.c	Display the complete student list.
stud_mod.c	Modify an existing student record.
stud_save.c	Save/load records and handle save-related operations.
student.h	Structure definition, declarations and function prototypes.

Note: The module names above are recommended. If a different module split is used, the same functional responsibilities must still be covered.

3. Student Record Structure

Each record should contain the following information:

```
struct student
{
    int rollno;
    char name[50];
    float percentage;
    struct student *next;
};
```

The **roll number** identifies a student record. The **next** pointer links one node to the next node in the singly linked list.

4. Main Menu

Display a menu similar to the following:

```
***** STUDENT RECORD MENU *****

a/A : Add new record
d/D : Delete a record
s/S : Show the list
```

m/M : Modify a record
v/V : Save records
e/E : Exit
t/T : Sort the list
l/L : Delete all the records
r/R : Reverse the list

Enter your choice:

The program should continue to display the menu until the user selects **e/E**.

5. Add New Record – stud_add.c

When the user selects **a/A**, call the add-record function.

- Create a new node dynamically using **malloc()**.
- Assign the **smallest positive integer that is not already used** as the roll number.
- The roll number must be unique.
- Ask the user to enter the student's name and percentage.
- Insert the new node into the singly linked list.

Example: If roll numbers 1, 2, 4 and 5 are present, the next roll number should be 3.

6. Delete Record – stud_del.c

When the user selects **d/D**, display:

R/r : Enter roll number to delete
N/n : Enter name to delete

Delete by roll number: Search for the specified roll number. If found, unlink the node, free its memory, and report successful deletion. If not found, display an appropriate message.

Delete by name: Search for matching names. If multiple records have the same name, display all matching records with their roll numbers so the user can identify the required record. The final deletion should be performed using the selected roll number.

7. Display Records – stud_show.c

When the user selects **s/S**, display the complete linked list in a clear tabular format.

```
-----  
Roll No. Name Percentage  
-----  
1 Rahul 78.50  
2 Priya 85.25  
-----
```

If the list is empty, display an appropriate message such as **No student records available**.

8. Modify Record – stud_mod.c

When the user selects **m/M**, display:

Enter which record to search for modification

R/r : Search by roll number
N/n : Search by name
P/p : Search by percentage

R/r: Search using the roll number. If found, display the current details and allow the user to enter the updated name and percentage. The roll number should remain unchanged.

N/n: Search using the name. If more than one record matches, display the matching records and ask for the roll number of the record to modify.

P/p: Search using the percentage. If more than one record matches, display the matching records and ask for the roll number of the record to modify.

If the requested record is not available, display an appropriate message.

9. Save and File Handling – stud_save.c

When the user selects **v/V**, save the complete student list into a file named **student.dat**.

The file should contain the student records so that saved data can be restored when the application is started again.

At program startup: If **student.dat** exists, load the saved records into the linked list. If it does not exist, start with an empty list.

10. Exit

When the user selects **e/E**, display:

S/s : Save and exit
E/e : Exit without saving

S/s – Save and Exit: Save the current records to student.dat, release all allocated nodes, and terminate the program.

E/e – Exit Without Saving: Do not save the current changes, release all allocated nodes, and terminate the program.

11. Sort Records – t/T

When the user selects **t/T**, display:

N/n : Sort with name
P/p : Sort with percentage

N/n: Sort the linked list alphabetically by student name.

P/p: Sort the linked list by percentage in **descending order** (highest percentage first).

The sorting operation should rearrange the linked-list nodes/links or otherwise correctly change the list order; the displayed result must reflect the selected sort.

12. Delete All Records – I/L

When the user selects **I/L**, delete every node from the linked list.

- Traverse the list node by node.
- Free each dynamically allocated node using **free()**.
- Set the head pointer to **NULL** after all nodes are deleted.
- Display an appropriate confirmation message.

Deleting all records from memory must not automatically delete student.dat. The file is affected only when the user explicitly saves.

13. Reverse the List – r/R

When the user selects **r/R**, reverse the singly linked list.

Before: 1 → 2 → 3 → 4 → NULL
After: 4 → 3 → 2 → 1 → NULL

The reversal should be performed by changing the linked-list pointers. Do not create a second set of nodes just to reverse the list.

14. Input Validation

- Percentage must be in the range **0.00 to 100.00**.
- Roll numbers must be positive and unique.
- Student name should not be empty.

- Handle invalid menu choices without terminating the program unexpectedly.
- Handle attempts to delete or modify a record that does not exist.

Students may add reasonable input validation beyond the above requirements, provided it does not change the specified functionality.

15. Memory Management Requirements

Dynamic memory management is an important part of this project.

- Use **malloc()** (or an equivalent dynamic-allocation approach allowed by the course) when creating a new node.
- Use **free()** whenever a node is deleted.
- Release all remaining nodes before the program terminates.
- Avoid memory leaks and invalid pointer access.

16. Functional Summary

Option	Required Functionality
a/A	Add a new dynamically allocated student record.
d/D	Delete by roll number or name.
s/S	Display the complete list in tabular form.
m/M	Modify a record by roll number, name or percentage.
v/V	Save records to student.dat.
e/E	Save & exit or exit without saving.
t/T	Sort by name or percentage.
l/L	Delete all nodes and free memory.
r/R	Reverse the singly linked list.

17. Expected Project Deliverables

- All required source files/modules.
- Header file containing the required declarations.
- Executable named **student**.
- The **student.dat** file after saving records.
- A short README/documentation file explaining compilation and execution steps.

The source code should be modular, readable and properly indented. Students should be able to explain the logic used for linked-list operations, dynamic memory allocation, file handling, searching, sorting and reversal during project evaluation.

18. Important Evaluation Expectations

The following are essential parts of the assignment and may be checked during execution and code review:

- Correct creation and linking of nodes.
- Correct deletion without breaking the linked list.
- Correct handling of duplicate names.
- Correct modification of the intended record.
- Correct saving and loading of student.dat.
- Correct sorting by name and percentage.

- Correct deletion of all nodes.
- Correct reversal of the list.
- Proper dynamic memory allocation and deallocation.
- Correct handling of empty-list and record-not-found cases.

End of Assignment